

P1 — les 3 verifications avant validation

Force Tracker — 10/09/2026 — version **ft-v1188**, deployee et verifiee verte (run #1060). Reponse a ton cahier « 3 verifications avant validation reelle ». Tout ce qui est chiffre ici a ete **execute**, pas deduit.

EN UNE PHRASE

Deux trous reels sur trois, et le troisieme est demontre sain. Le point 1 n'etait pas « non teste », il **n'existait pas** — et il cachait une regression pire que ce que tu visais. Le point 2 decrit un cas que l'application ne sait pas produire, mais la question a fait tomber autre chose. Le point 3 n'a aucune derive cumulative, et pourtant l'arrondi coutait vraiment.

COMMENT LIRE LES CHIFFRES

Chaque mesure vient d'un **vrai navigateur** (Chromium + Playwright) qui appelle les vraies fonctions et relit ce que l'application **ecrit** dans son journal. Aucune valeur n'est calculee a la main pour ce document. **Limites** : pas d'accès aux donnees reelles de Michel, et **pas de WebKit** ici — le rendu iPhone reste a valider par lui.

1. Corriger portionWeightG apres coup

Ton cahier dit : « *le document P1 indique explicitement que ce chemin n'est pas teste* ». **Mesure : il n'existait pas.** Aucun champ de l'ecran d'edition ne touchait portionLabel ni portionWeightG.

ET LA MESURE A TROUVE PIRE QUE CE QUE TU CHERCHAIS

Sonde sur une ligne q:2, u: 'portion', portionWeightG:125 avec un pour-100 g : l'ecran d'edition affichait « **Quantite (g) = 250** », et **enregistrer sans rien toucher** reecrivait la ligne en q:250, u: 'g'. C'est mot pour mot ce que Michel refuse au point 2 de ses decisions : « *on perd alors l'information 2 steaks* ». Et portionLabel / portionWeightG **survivaient a cote d'un** u: 'g' — deux sources qui se contredisent, la famille de bugs que cette consigne nommait justement.

LA CAUSE VAUT PLUS QUE LE CORRECTIF — C'EST LA JUMELLE D'UNE VERSION QUE J'AI LIVREE HIER

En **ft-v1186** j'avais corrige exactement ce default — « *le pour-100 g ne decide plus de l'unite* » — sur les **deux** portes de l'ecran d'**ajout**, avec temoin de non-regression et 15 mutations toutes mordantes. **L'ecran d'EDITION faisait la meme chose, dans la meme version, a 900 lignes d'ecart.** Ce qui rend cette famille couteuse : elle ne se voit **ni a la relecture ni au banc d'essai**. Le code corrige est juste, les temoins verts le meritent, le controle negatif mord. Tout ce qui mesure dit vrai — et la moitié du bug est encore la. **8e fois recensee** entre les fonctions `_af*` et `_ef*` ; ecrite comme famille `BUGS.md` 59.

Le correctif : deux moitiés

a) le pour-100 g ne decide plus de l'unite, ici non plus. Le test porte sur `e.u`, donc un produit scanne (u: 'g') garde **exactement** son champ grammes — rien ne change pour lui, et deux temoins le figent.

```
const ancre = (e.per100 && e.u!=='portion') ? null
  : (e.q>0 ? {v:e.q,u:e.u||'g',src:'quantité enregistrée'}
    : (mNom ? {v:parseFloat(mNom[1].replace(',','.')),u:mNom[2].toLowerCase(),src:'lu dans le nom'} : null));
if(e.per100 && e.u!=='portion'){
```

b) deux champs pour corriger la definition, et le pour-100 g qui **suit** la nouvelle definition.

Le point delicat, et c'est lui qui valait la mesure

Ta regle est : « *une definition de portion qui change doit invalider ou recalculer toute donnee derivee de l'ancienne definition* ». Appliquee telle quelle, elle casse autre chose :

strategie	ce qu'elle produit
rederiver toujours	ecrase le pour-100 g publie d'un produit scanne des qu'on touche au poids — une valeur mesuree remplacee par une declaration
ne jamais rederiver	un pour-100 g calcule pour 125 g survit a un passage a 150 g — ce que tu interdis

ON NE DEVINE PAS, ON VERIFIE

Un pour-100 g **derive** vaut, par construction, totaux $\times 100 / (q \times \text{poids})$. On refait donc le calcul avec l'**ancien** etat : s'il retombe dessus, il en venait, donc il doit suivre ; sinon il vient d'ailleurs et on n'y touche pas. **C'est la redondance interne du document**, pas une supposition sur la provenance.

```
function _per100SuitLaPortion(av){
  if(!av.per100) return true;
  const m=(+av.q||0)*(+av.pw||0);
  if(!(m>0)) return false;
  const f=100/m, ok=(a,b)=>Math.abs((+a||0)-(+b||0))<=0.6;
  return ok(av.per100.kcal,(+av.kcal||0)*f) && ok(av.per100.prot,(+av.prot||0)*f)
    && ok(av.per100.carbs,(+av.carbs||0)*f) && ok(av.per100.fat,(+av.fat||0)*f);
}
```

Ce que la sonde rend, apres correctif

geste	resultat mesure
ouvrir « 2 steaks de 125 g »	reste en portions : ef-prop=2, ef-pnom=steak, ef-ppoids=125 ; l'ecran dit « 1 steak (125 g) · 250 kcal. Soit 250 g en tout. »
taper 150 dans le poids	la definition se met a jour pendant la frappe : « 1 steak (150 g) ... Soit 300 g en tout » — sans detruire le champ
enregistrer	portionWeightG=150 et per100 = 166,7 / 13,3 / 0 / 12 — la valeur exacte (500/3 = 166,667). q=2, u='portion', label intacts, macros inchangees
ouvrir + enregistrer sans rien toucher	rien ne change — c'etait le defaut d'origine
un produit scanne (u: 'g')	garde son champ grammes (200 g) et son pour-100 g publie (130/26/0/3)
un scan en portions dont le pour-100 g contredit la definition	on ajoute un poids de 125 g -> le pour-100 g publie reste 97/4/11,3/3,2 , intact
vider le champ poids	portionWeightG retire, et le pour-100 g qui en dependait aussi — aucune verite orpheline

2. Deux favoris portant le meme nom

Tu soupconnes une contamination via `_majDefFavori()`, qui retrouve le favori par name. **La reponse tient en deux temps, et le premier n'etait pas evident.**

LE DOUBLON EST IMPOSSIBLE A CREER — MESURE PAR LA VRAIE PORTE

`toggleFavFood` cherche `name.toLowerCase()` et **retire** au lieu d'ajouter quand il trouve. Trois clics d'etoile d'affilee donnent **1 favori, puis 0, puis 1** — casse comprise (« Steak hache » et « STEAK HACHE » sont la meme cle). Et une restauration cloud **remplace** la liste en bloc au lieu de concatener. **Le nom EST l'identite d'un favori dans cette application, par construction, pas par hasard.**

Ta consigne dit : « ne pas inventer un nouvel identifiant si un identifiant fiable existe deja ». **Mesure : il n'en existe aucun.** Un favori stocke name / kcal / prot / carbs / fat / per100 / q / u / portionLabel / portionWeightG — **pas de** sourceId, pas de code-barres. En ajouter un serait un champ sans producteur, pour un etat que l'application ne sait pas fabriquer.

MAIS .find() REND LE PREMIER — ET LA, TU AS RAISON

Force a la main l'etat impossible (deux favoris de meme nom, B en premier), puis modifie A : **les 150 g atterrisaient sur B**. Mesure. Le correctif n'invente pas d'identifiant : **on refuse d'agir sur une identite ambigue**. Une ligne, un temoin. *Ne rien faire vaut mieux qu'agir sur la mauvaise ligne.*

```
const k=String(e.name||'').toLowerCase(); if(!k) return;
if(S.savedFoods.filter(x=>String(x.name||'').toLowerCase()===k).length>1) return;
const f=S.savedFoods.find(x=>String(x.name||'').toLowerCase()===k); if(!f) return;
```

Verifie apres correctif : le meme geste laisse **B a 100 g et A a 125 g**, les deux intacts.

Et la question a fait tomber autre chose

LE FAVORI NE SUIVAIT PAS UNE DEFINITION CORRIGEE DEPUIS L'EDITION

Mesure : corriger « 1 steak = 150 g » dans l'ecran d'edition laissait le favori a **125 g**. C'est exactement la vieille copie silencieuse que la decision 5 de Michel voulait empecher — `_majDefFavori` n'etait appelee qu'a l'**ajout**. La **porte jumelle, encore** : aucun mecanisme neuf, un appel. Verifie : le favori passe a 150 g / « pave », et **ses macros ne bougent pas** (favori 250/20, repas 500/40).

3. La precision du per100 derive

Tu demandes : « *si l'arrondi n'a aucun effet persistant sur la donnee, le demontrer* ». Il n'en a aucun, et voici la demonstration.

AUCUNE DERIVE CUMULATIVE — 10 CYCLES REELS

Une ligne saisie (1 part de 140 g, 355/30/44/6), puis **reprise et re-enregistree neuf fois** par la vraie porte « Mes aliments ». Empreinte des cycles 1, 2, 5 et 10 : **identique au caractere pres**. La raison est structurelle : le pour-100 g se redérive toujours des **TOTAUX**, jamais du pour-100 g precedent. **C'est un point fixe, pas une cascade.**

Mais l'arrondi entier coutait vraiment, en aval. Meme cas, une fois enregistre :

	kcal	prot	carbs	fat
pour-100 g exact	253,57	21,43	31,43	4,29
arrondi entier (avant)	254	21	31	4
puis redemander 280 g	711	59	87	11
le double exact	710	60	88	12

Soit -8,3 % sur les lipides. Plus la macro est petite, plus l'erreur relative est grosse.

ET LA CORRECTION EXISTAIT DEJA — ELLE N'ETAIT PAS POSEE ICI

Ta consigne dit : « *corriger avec le niveau de precision deja utilise ailleurs* ». Il y en a un : `_per100d1` (une decimale), qui sert **7 portes** depuis ft-v1170 — Open Food Facts, CIQUAL, les marques, l'etiquette photo, le scan. Les **deux** pour-100 g **derives** de `_provFood` etaient restes en `Math.round`. La **8e porte** — **meme famille que le point 1**. Et ca ne change rien a l'ecran : `_qtyResca` le arrondit deja les 4 champs a l'entier. La decimale ne sert qu'a ce qui est **conserve et re-multiplie**.

```
p.per100={kcal:_per100d1((+vals.kcal|0)*f), prot:_per100d1((+vals.prot|0)*f),
  carbs:_per100d1((+vals.carbs|0)*f), fat:_per100d1((+vals.fat|0)*f)};
```

Apres : 253,6 / 21,4 / 31,4 / 4,3, et 280 g rendent **710 / 60 / 88 / 12** — exact.

Ce que la mesure a trouve et que tu ne visais pas

La **passee complete a rougi sur trois temoins plus anciens** — un de ft-v1051, deux de ft-v1162. Tous les trois figeaient `Math.round`, pas leur garantie. Le cas de ft-v1162 merite d'etre lu en entier, parce que **la decimale y a l'air moins precise** : la reprise a 180 g affiche **129** au lieu de **130**.

	valeur
verite d'origine (274 kcal / 380 g, x180)	129,79
ce que l'application stocke	79 kcal pour 110 g — les totaux sont des entiers
depuis cette donnee : 79/110 x 180	129,27
chemin entier (per100 = 72)	129,60 -> 130 — plus proche de 129,79 par chance
chemin decimal (per100 = 71,8)	129,24 -> 129 — fidele a ce qui est reellement stocke

LE VRAI PLANCHER DE PRECISION N'EST PAS LE per100, CE SONT LES TOTAUX ENTIERS

Le 130 venait d'une erreur d'arrondi qui pointait dans le bon sens. L'application ne **peut plus** connaître 129,79 : le 79 a été arrondi **avant** d'être écrit (parseInt sur les 4 champs). **129 est la seule réponse juste au regard de la donnée.** Affiner le pour-100 g ne peut donc pas aller plus loin. Le seul gain restant serait de stocker les **totaux** avec une décimale — ce qui touche la saisie, le rescale, l'affichage, l'export CSV et le cloud. **Chantier a part, non décidé.** Les interdits que ces témoins figent (**249, 448, 780**) n'ont pas bougé, et les trois ont été **éprouvés** : la mutation qui rétablit le défaut d'origine les fait rougir.

Un piège d'outillage, dit parce qu'il resservira

Ma première mesure du point 2 concluait « impossible de créer deux favoris » — **vrai, mais pour la mauvaise raison.** toggleFavFood finit par _renderFoodQuickList(), qui **reconstruit** la liste : ma fixture posée à la main était écrasée, et mon deuxième clic tapait dans le vide. **Un test qui n'emploie pas le schéma de la production ne teste rien, il rassure.** Refait en reposant la fixture avant chaque clic — la conclusion tient, la mesure aussi.

Contrôle négatif — une mutation par protection

Ton point 4 : « chaque mutation doit faire mordre un témoin précis ; une mutation qui ne mord pas doit être expliquée ». **15 mutations, toutes mordent**, et aucune n'a eu besoin d'explication.

mutation	rouges
l'unité volée par le pour-100 g	8
la branche grammes qui reprend la main	10
les deux champs de définition retirés	6
le pour-100 g qui ne suit plus la nouvelle définition	2
rederiver a l'aveugle (le scan se fait écraser)	1 — exactement le scan protège
le refus d'agir sur une identité ambiguë retire	1
le favori qui ne suit plus une correction d'édition	1
arrondi entier cote portion	1
arrondi entier cote grammes	1
la frappe qui ne met plus à jour la définition	1
les macros du favori écrasées	1
le poids retire qui laisse un pour-100 g orphelin	1
la définition non hydratée à l'ouverture	4
(témoin resserre) division par la référence au lieu de l'affichée	2
(témoin resserre) le défaut d'origine de ft-v1162	3

Non-regression

Suite complète : **parcours 3481 verts, 0 rouge(s)**, calculs 339/339, muscles 241/241, croises 50/50, dates 9/9, données classées 0 trou. Les blocs que tu listes (CCLXXVII, CCLXXVIII, CLXVIII, CCLXXIX, CCLXXX, plus celui de ft-v1186) sont **tous verts**. Nouveau bloc **CCLXXXV**, 18 témoins.

Ce qui reste ouvert — dit, pas taise

trou	pourquoi il reste
une ligne sans aucune quantite ne peut pas recevoir de definition	l'ecran d'edition a trois etats ; je n'ai ouvert les champs que dans celui ou $q > 0$ avec $u: 'portion'$. L'etat « boutons » écrit pourtant $q: n$, $u: 'portion'$ a l'enregistrement. Non elargi expres : ton perimetre dit « ne touche a rien d'autre »
une etiquette fausse ne se retire que par l'edition	les puces basculent et le champ libre se vide, mais c'est moins evident qu'a l'edition — juge humain , a regarder par Michel
la tolerance de 0,6 de <code>_per100SuitLaPortion</code> est un choix	un pour-100 g publie qui tomberait par hasard a 0,6 pres de ce que la definition produirait serait traite comme derive. Probabilite faible, consequence reelle. Le remede propre serait une provenance ecrite sur le per100 — S. foodLog ne stocke aujourd'hui ni source ni version pour ce champ
le plancher des totaux entiers	detaille plus haut ; chantier a part, non decide
le journal du jour n'affiche aucune quantite	ni « 2 steaks » ni « 250 g » — c'est le chantier 6 de Michel, separe expres

PERIMETRE INTERDIT — RESPECTE

Rien touche a : migration des 17 jours, cru/cuit, recherche CIQUAL, `alias.json`, scan code-barres, l4, `visualViewport`, Milo, programmes, debriefs de seance.

VERSION ET COMMIT

ft-v1188 — commit fb392db4 sur master. Deploiement **verifie vert** : run #1060, conclusion: success a **08:52:22 UTC**. Ni backend ni worker attendus (`Code.js` / `worker.js` non touches). **Aucun point n'est reste ambigu** — c'etait ta condition de deploiement.

UNE CHOSE A SAVOIR SI MICHEL COMPARE DEUX COPIES D'ECRAN

Une meme ligne reprise peut afficher **1 kcal de moins** qu'avant ft-v1188. Ce n'est pas une perte : c'est l'arrondi de chance qui disparaît.